An LLM acts like a brilliant student taking a closed-book exam based entirely on memory. RAG transforms that same student into an open-book test taker by handing them a relevant textbook right before they write the answer.

In [1]:
```
try:
    import torch
    import pypdf
    print(f"✅ Success! torch version: {torch.__version__}")
    print(f"✅ Success! pypdf version: {pypdf.__version__}")
except ImportError as e:
    print(f"❌ Error: {e}")
```

```
✅ Success! torch version: 2.11.0+cu128
✅ Success! pypdf version: 6.14.2
```

In [2]:
```
import torch
torch.cuda.is_available()
```

Out[2]: True

In [3]:
```
# Updated cell 2

import os
import json

# Your exact local file paths
KB_FILE = "knowledge_base.json"
EVAL_FILE = "evaluation_set.json"

# Check if the files exist before reading
if not os.path.exists(KB_FILE) or not os.path.exists(EVAL_FILE):
    raise FileNotFoundError(
        f"Ensure '{KB_FILE}' and '{EVAL_FILE}' containing your Venezuela/CBI "
        f"data are placed in this exact folder directory: {os.getcwd()}"
    )

# Load your actual data assets
with open(KB_FILE, "r", encoding="utf-8") as f:
    kb_data = json.load(f)
with open(EVAL_FILE, "r", encoding="utf-8") as f:
    eval_data = json.load(f)

print("==== DATA ACQUISITION SUCCESS ====")
print(f"Loaded {len(kb_data)} knowledge chunks (Fields parsed: 'chunk_id', 'text
print(f"Loaded {len(eval_data)} evaluation items (Fields parsed: 'user_query', '
```

```
==== DATA ACQUISITION SUCCESS ====
Loaded 19 knowledge chunks (Fields parsed: 'chunk_id', 'text', 'metadata')
Loaded 17 evaluation items (Fields parsed: 'user_query', 'ground_truth_answer')
```

In [4]:
```
# Upgraded Cell 3: Word-Level Vocabulary Engine, 3rd Iteration

# Compile every distinct word across your real text files to build a word-index
raw_corpus = ""
for item in kb_data: raw_corpus += item["text"] + " "
for item in eval_data: raw_corpus += item["user_query"] + " " + item["ground_tru

# Clean basic punctuation to extract raw words
```

```
processed_words = []
for word in raw_corpus.lower().split():
    processed_words.append(word.strip(".,;:!?()\""))

# Remove duplicates and register special token flags
unique_words = sorted(list(set(processed_words)))
vocab = ["<pad>", "<sos>", "<eos>", "<unk>"] + unique_words

# Construct word conversion mappings
word_to_id = {w: i for i, w in enumerate(vocab)}
id_to_word = {i: w for i, w in enumerate(vocab)}

VOCAB_SIZE = len(vocab)
PAD_IDX = word_to_id["<pad>"]
SOS_IDX = word_to_id["<sos>"]
EOS_IDX = word_to_id["<eos>"]
UNK_IDX = word_to_id["<unk>"]

print(f"Total Unique Vocabulary Words Found: {len(unique_words)}")
print(f"Total Matrix Input Size (Vocab + Controls): {VOCAB_SIZE}")

def encode_text(text, max_len=128):
    """Translates strings into a list of word vocabulary numerical IDs."""
    tokens = [SOS_IDX]
    for word in text.lower().split():
        word = word.strip(".,;:!?()\"")
        tokens.append(word_to_id.get(word, UNK_IDX)) # Fallback to <unk> if wora
    tokens.append(EOS_IDX)

    if len(tokens) < max_len:
        tokens += [PAD_IDX] * (max_len - len(tokens))
    return tokens[:max_len]

# Run a sample validation slice
print(f"\nVerification Test: 'Operation Chakra-VI' -> {encode_text('Operation Ch
```

```
Total Unique Vocabulary Words Found: 663
Total Matrix Input Size (Vocab + Controls): 667

Verification Test: 'Operation Chakra-VI' -> [1, 428, 103, 2, 0, 0]
```

In [5]:

```
# updated cell 4

def manual_jaccard_retriever(query, kb_list):
    """A zero-dependency search engine tracking maximum word intersection metric
    q_words = set(query.lower().split())
    best_chunk = kb_list[0]["text"] if len(kb_list) > 0 else ""
    best_score = -1

    for item in kb_list:
        # Maps natively to your document text data stream layer
        chunk_words = set(item["text"].lower().split())
        intersection = q_words.intersection(chunk_words)
        union = q_words.union(chunk_words)
        score = len(intersection) / len(union) if len(union) > 0 else 0

        if score > best_score:
            best_score = score
            best_chunk = item["text"]
```

```
    return best_chunk

# Test-run the retriever with one of your actual evaluation query types
sample_query = "What is the name of the CBI operation against digital arrest sca
matched_context = manual_jaccard_retriever(sample_query, kb_data)

print(f"Query Test      : {sample_query}")
print(f"Retrieved Block : {matched_context[:120]}...")
```

```
Query Test      : What is the name of the CBI operation against digital arrest sc
ams?
Retrieved Block : As part of Operation Chakra-VI, the CBI arrested two key accuse
d: Naresh from Chennai, Tamil Nadu, and Sanjib Saha from ...
```

In [6]:

```
# Updated cell 5

import torch
from torch.utils.data import Dataset, DataLoader

class RAGDataset(Dataset):
    """Maps custom text formats to coupled, padded training tensor arrays."""
    def __init__(self, eval_set, kb_set, max_len=128):
        self.samples = []
        for item in eval_set:
            # 1. Access your real query key string: item["user_query"]
            query = item["user_query"]

            # 2. Execute local retrieval search pass
            context = manual_jaccard_retriever(query, kb_set)

            # 3. Stitch prompt structures into a cohesive sequence matrix layout
            input_text = f"Context: {context} | Query: {query} -> Answer: "

            # 4. Access your real output key target string: item["ground_truth_a
            target_text = item["ground_truth_answer"]

            self.samples.append({
                "input_ids": torch.tensor(encode_text(input_text, max_len), dtyp
                "target_ids": torch.tensor(encode_text(target_text, max_len), dt
            })

    def __len__(self):
        return len(self.samples)

    def __getitem__(self, idx):
        return self.samples[idx]

# Instantiate dataset and batch loaders using your actual variables
dataset = RAGDataset(eval_data, kb_data)
dataloader = DataLoader(dataset, batch_size=2, shuffle=True)

print(f"Successfully constructed PyTorch DataLoader around your custom JSON keys
print(f"Total training data vectors loaded into grid: {len(dataset)}")
```

```
Successfully constructed PyTorch DataLoader around your custom JSON keys.
Total training data vectors loaded into grid: 17
```

In [7]:

```
# Updated Cell 6: 4th Iteration

import torch.nn as nn
```

```
class MiniCausalTransformer(nn.Module):
    """
    A pure PyTorch Causal Language Transformer Model designed from scratch.
    Expanded dimensions to force memorisation on localized data profiles.
    """
    def __init__(self, vocab_size, embedding_dim=128, num_heads=4, hidden_dim=25
        super(MiniCausalTransformer, self).__init__()
        # 1. Token Embedding Matrix Lookup Array
        self.token_embedding = nn.Embedding(vocab_size, embedding_dim)
        # 2. Spatial Index Tracking Position Layer
        self.position_embedding = nn.Embedding(max_len, embedding_dim)

        # 3. Structural Multi-Head Self-Attention Core (4 Layers Deep)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embedding_dim,
            nhead=num_heads,
            dim_feedforward=hidden_dim,
            batch_first=True,
            norm_first=True
        )
        self.transformer_block = nn.TransformerEncoder(encoder_layer, num_layers

        # 4. Projection Layer: Maps multi-dimensional outputs back to word vocab
        self.fc_out = nn.Linear(embedding_dim, vocab_size)

    def forward(self, x):
        batch_size, seq_len = x.shape
        positions = torch.arange(0, seq_len).expand(batch_size, seq_len).to(x.de

        # Merge semantic token arrays with spatial position tracking layers
        out = self.token_embedding(x) + self.position_embedding(positions)

        # Generate a lower-triangular causal mask to block future token sightlin
        mask = nn.Transformer.generate_square_subsequent_mask(seq_len).to(x.devi

        out = self.transformer_block(out, mask=mask, is_causal=True)
        return self.fc_out(out)

# ======================================================================
# MODEL INITIALIZATION & HARDWARE COUPLING
# ======================================================================
# Set high-capacity hyper-parameters to scale model parameters
EMBEDDING_DIM = 128
NUM_HEADS = 4
HIDDEN_DIM = 256

# Instantiate the custom network layers
model = MiniCausalTransformer(
    vocab_size=VOCAB_SIZE,
    embedding_dim=EMBEDDING_DIM,
    num_heads=NUM_HEADS,
    hidden_dim=HIDDEN_DIM,
    max_len=128
)

# Detect if your laptop graphics card driver setup completed successfully
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

print(f"Model mounted cleanly on compute layer: {device}")
```

```
Model mounted cleanly on compute layer: cuda
C:\Users\lanst\AppData\Local\Temp\ipykernel_24312\995997839.py:25: UserWarning: e
nable_nested_tensor is True, but self.use_nested_tensor is False because encoder_
layer.norm_first was True
  self.transformer_block = nn.TransformerEncoder(encoder_layer, num_layers=4)
```

In [8]:
```python
# Just checking if your GPU is available and recognized by PyTorch

import torch
print("Is GPU available?:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU Card Name   :", torch.cuda.get_device_name(0))
```

```
Is GPU available?: True
GPU Card Name   : NVIDIA GeForce RTX 5050 Laptop GPU
```

Cell 7

In [ ]:
```python
# # Updated Cell 7: 3rd Iteration, 4th Iteration

# import torch.optim as optim

# # Categorical Cross Entropy handles multi-word prediction paths
# criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
# # Adam optimizer scales step matrices
# # Iteration 4
# optimizer = optim.Adam(model.parameters(), lr=0.001)



# print("Starting custom model optimization training loops...")
# # epochs = 300 # Run optimization 300 times to converge weights
# # Iteration 4
# epochs = 1000 # Run optimization 1000 times to converge weights

# model.train()
# for epoch in range(epochs):
#     epoch_loss = 0
#     for batch in dataloader:
#         inputs = batch["input_ids"].to(device)
#         targets = batch["target_ids"].to(device)

#         optimizer.zero_grad()
#         logits = model(inputs)

#         loss = criterion(logits.view(-1, VOCAB_SIZE), targets.view(-1))
#         loss.backward()
#         optimizer.step()

#         epoch_loss += loss.item()

#     if (epoch + 1) % 50 == 0:
#         print(f"Epoch [{epoch+1}/{epochs}] | Parameter Error Rate (Loss): {epo

# print("\nModel weights successfully calibrated and locked.")
```

In [9]:
```python
# Cell 7: 5th Iteration

import torch.optim as optim
```

```
import torch.nn as nn

# Categorical Cross Entropy handles multi-word prediction paths
# Ignore pad index so your padding fields don't corrupt gradient updates
criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
optimizer = optim.AdamW(model.parameters(), lr=0.0005) # AdamW prevents extreme

print("Starting custom model optimization training loops...")
epochs = 400 # Overfitting local parameters requires a steady convergence baseli

model.train()
for epoch in range(epochs):
    epoch_loss = 0
    for batch in dataloader:
        # Pull your coupled auto-regressive text sequence vectors
        # Shape: [Batch, Seq_Len]
        tokens = batch["input_ids"].to(device)

        # Causal Token Shift: Inputs are 0 to N-1 | Targets are 1 to N
        inputs = tokens[:, :-1]
        targets = tokens[:, 1:]

        optimizer.zero_grad()
        logits = model(inputs)

        # Flatten tensors for cross-entropy verification
        loss = criterion(logits.view(-1, VOCAB_SIZE), targets.reshape(-1))
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()

    if (epoch + 1) % 50 == 0:
        print(f"Epoch [{epoch+1}/{epochs}] | Parameter Error Rate (Loss): {epoch

print("\nModel weights successfully calibrated and locked to RAG context paths."
```

```
Starting custom model optimization training loops...
Epoch [50/400] | Parameter Error Rate (Loss): 0.0608
Epoch [100/400] | Parameter Error Rate (Loss): 0.0504
Epoch [150/400] | Parameter Error Rate (Loss): 0.0504
Epoch [200/400] | Parameter Error Rate (Loss): 0.0468
Epoch [250/400] | Parameter Error Rate (Loss): 0.0464
Epoch [300/400] | Parameter Error Rate (Loss): 0.0427
Epoch [350/400] | Parameter Error Rate (Loss): 0.0405
Epoch [400/400] | Parameter Error Rate (Loss): 0.0478

Model weights successfully calibrated and locked to RAG context paths.
```

Cell 8

In [ ]:

```
# # Updated cell 8: 4th Iteration

# # Iteration 4

# import torch
# import torch.nn.functional as F

# # Put the model into evaluation mode (disables dropout layers)
# model.eval()
```

```
# # Input whatever validation question you want to query against your model
# user_question = "What are the locations of the accused with respect to the dig

# print(f"Evaluating User Query Sequence: '{user_question}'")
# print("Searching custom index vectors...")

# # 1. LOCAL RETRIEVAL: Search your JSON knowledge base for the most relevant da
# retrieved_context = manual_jaccard_retriever(user_question, kb_data)

# # 2.BluePRINT STRUCTURING: Reassemble the exact text layout template used duri
# inference_input = f"context: {retrieved_context} | query: {user_question} -> a
# input_tokens = encode_text(inference_input)

# # Convert the word index list into a PyTorch Tensor matrix and push it to your
# input_tensor = torch.tensor([input_tokens], dtype=torch.long).to(device)

# generated_words = []

# # 3. AUTOREGRESSIVE GENERATION LOOP: Predict next tokens step-by-step
# with torch.no_grad():
#     for _ in range(40): # Cap generation output at 40 words max
#         # Run forward pass matrix math across your transformer layers
#         logits = model(input_tensor)

#         # Read the raw output scores for the very last word position in the se
#         next_token_logits = logits[0, -1, :]

#         # RIGID ARGMAX DECODING: Pick the absolute highest scoring word in the
#         # This completely disables randomness to enforce deterministic, factua
#         predicted_token_id = torch.argmax(next_token_logits).item()

#         # Terminate generation immediately if the model throws the End-of-Stri
#         if predicted_token_id == EOS_IDX:
#             break

#         # Translate the numerical ID back into a clean string word
#         word = id_to_word.get(predicted_token_id, "")

#         # Prevent training boundary tags from leaking into your final answer s
#         if word not in ["<pad>", "<sos>", "<unk>"]:
#             generated_words.append(word)

#         # Concat the newest token ID back into your active tensor input line
#         # This dynamically updates your sliding context window for the next it
#         new_token_tensor = torch.tensor([[predicted_token_id]], dtype=torch.lo
#         input_tensor = torch.cat((input_tensor, new_token_tensor), dim=1)[:, :

# # Clean up final string list spacing layout adjustments
# final_answer = " ".join(generated_words)

# print("\n================== CUSTOM SYSTEM RAG SEARCH OUTPUT ==================
# print(f"Retrieved Document Context: \"{retrieved_context[:120]}...\"")
# print(f"Generated Language Response: \"{final_answer}\"")
# print("======================================================================")
```

In [10]:

```
# Cell 8: 5th Iteration

import torch
import torch.nn.functional as F
```

```
model.eval()

# Your validation query type
user_question = "What are the locations of the accused with respect to the digit
print(f"Evaluating User Query Sequence: '{user_question}'")

# 1. RAG STEP: Execute Jaccard text retrieval matching from your JSON database m
retrieved_context = manual_jaccard_retriever(user_question, kb_data)

# 2. PROMPT SETUP STEP: Assemble the exact textual sequence framework the model
inference_input = f"Context: {retrieved_context} | Query: {user_question} -> Ans

# Clean inline encoding to build our dynamic sequence array
input_tokens = [SOS_IDX] + [word_to_id.get(w.strip(".,;:!?()\""), UNK_IDX) for w
prompt_length = len(input_tokens) # Lock prompt boundary index

generated_words = []

# 3. LLM STEP: Generates words one by one using a sliding context matrix
with torch.no_grad():
    for _ in range(40):
        # Format active sequence slice into PyTorch execution tensor
        input_tensor = torch.tensor([input_tokens], dtype=torch.long).to(device)

        # Run forward pass matrix math across your transformer layers
        logits = model(input_tensor)

        # Extract prediction scores for the absolute last word token position
        next_token_logits = logits[0, -1, :]

        # Rigid argmax decoding path
        predicted_token_id = torch.argmax(next_token_logits).item()

        # Terminate if the model signals sentence completion
        if predicted_token_id == EOS_IDX:
            break

        # Append token to active history so the model remembers what it just sai
        input_tokens.append(predicted_token_id)

        # Decode token value back into readable string array elements
        word = id_to_word.get(predicted_token_id, "<unk>")
        if word not in ["<pad>", "<sos>", "<eos>"]:
            generated_words.append(word)

final_answer = " ".join(generated_words)

print("\n================== CUSTOM SYSTEM RAG SEARCH OUTPUT ==================")
print(f"Retrieved Document Context: \"{retrieved_context[:120]}...\"")
print(f"Generated Language Response: \"{final_answer}\"")
print("=====================================================================")
```

```
Evaluating User Query Sequence: 'What are the locations of the accused with respe
ct to the digital arrests conducted by the CBI?'

================== CUSTOM SYSTEM RAG SEARCH OUTPUT ==================
Retrieved Document Context: "As part of Operation Chakra-VI, the CBI arrested two
key accused: Naresh from Chennai, Tamil Nadu, and Sanjib Saha from ..."
Generated Language Response: "<unk>"
======================================================================
```